

Fatima Jinnah Women University
Department of Computer Sciences

Lab 04: Simple Python Database Connections

Course	DevOps (CS-649 Cloud DevOps)
Instructor	Sir Waqas Saleem
Student Name	Kiran Siddique
Registration No.	2023-BCS-052
Section	A
GitHub Username	KiranSiddique
Submission Repository	DevOps (Labs/Lab04)
Date of Submission	26 October 2026

Table of Contents

Task 1: Fork and Clone the Databases Repository 2
Task 2: Install Docker, Docker Compose, and Python on Ubuntu 3
Task 3: Start the Database Services 4
Task 4: MySQL with Values in the Code 5
Task 5: MongoDB with Values in the Code 5
Task 6: MongoDB with os.getenv() 6
Task 7: MongoDB with a Credentials File 7
Exam Questions 9
Summary 10

Lab Overview

Item	Detail
Lab	Lab 04: Simple Python Database Connections
Fork URL	https://github.com/KiranSiddique/Databases
Submission Directory	DevOps / Labs/Lab04
VM	Ubuntu Server, user kiransiddique, hostname ubuntu
Docker	29.1.3
Docker Compose	v5.6.1
Git	2.53.0
MySQL	8.4
MongoDB	7.0 (used instead of 8.0 with the instructor's permission)

Task 1: Fork and Clone the Databases Repository

I opened the instructor's Databases repository on GitHub and forked it under my own account, keeping the name *Databases*. The page shows **KiranSiddique/Databases** and the note "forked from WaqasSaleem97/Databases", which confirms that it is my own fork. The screenshot is split-screen, with the GitHub page on the left and my Ubuntu terminal (`whoami` and `hostname`) on the right.

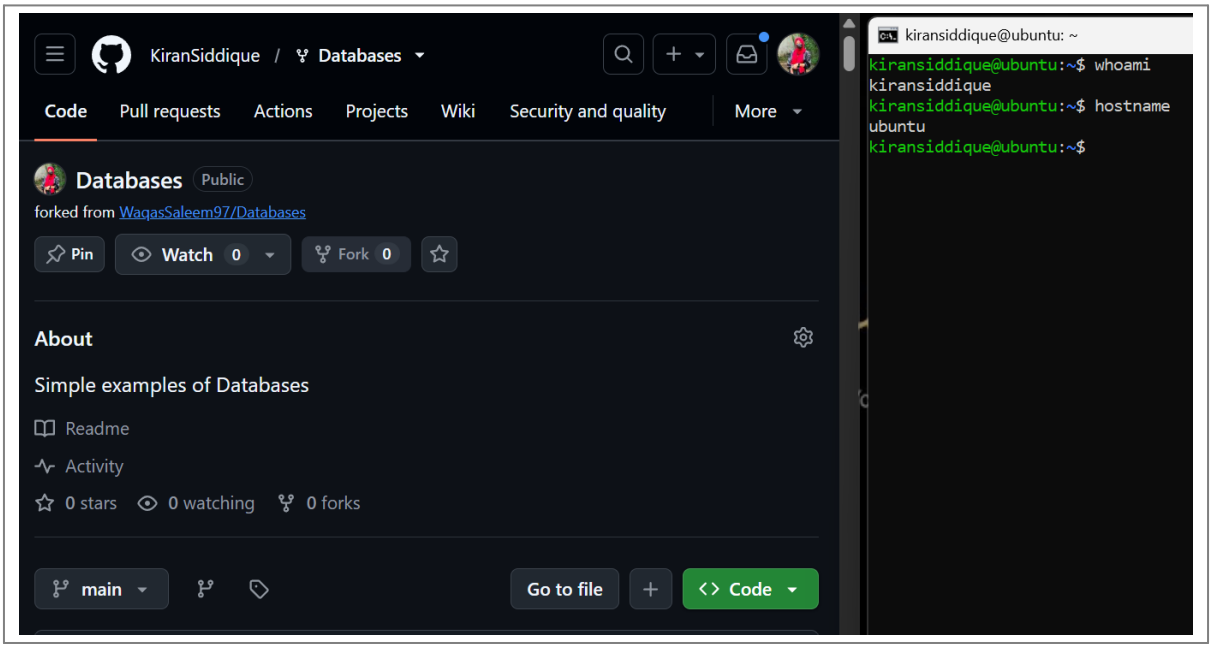


Figure 1: Fork of Databases on GitHub with the terminal identity (`databases_fork.png`)

Next I cloned my own fork over SSH into my home directory and listed its contents. The clone contains `db_connect.py`, `compose.yaml`, `credentials.ini.example`, `requirements.txt` and the `db-init` folder.

```
cd ~
git clone git@github.com:KiranSiddique/Databases.git
cd Databases
ls
```

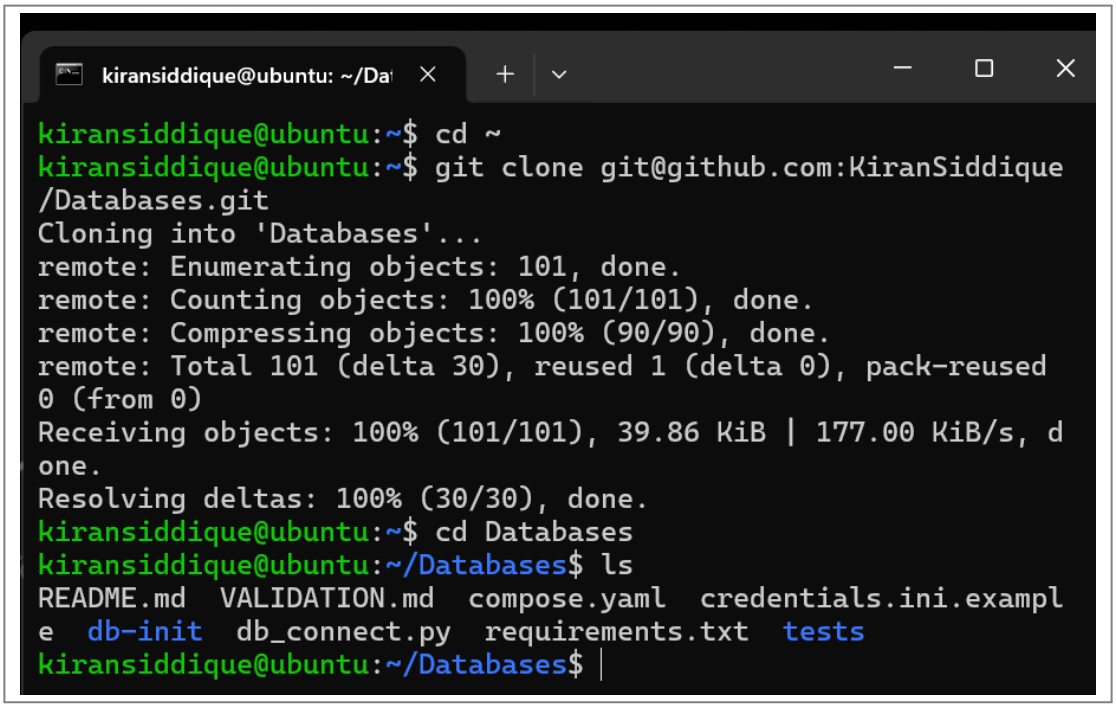


Figure 2: Cloning my fork of Databases (`databases_clone.png`)

Task 2: Install Docker, Docker Compose, and Python on Ubuntu

On the Ubuntu server I installed Docker Engine (the `docker.io` package) and curl, then downloaded the Docker Compose plug-in into the system-wide CLI plug-in directory and made it executable. I started the Docker service, added my account to the `docker` group, signed out completely and reconnected so the new group membership took effect. After reconnecting, Git, Docker and Docker Compose all reported their versions, and `groups` lists *docker*.

```
sudo apt update -y
sudo apt install -y docker.io curl
sudo mkdir -p /usr/local/lib/docker/cli-plugins
sudo curl -SL https://github.com/docker/compose/releases/latest/download/docker-compos
  -o /usr/local/lib/docker/cli-plugins/docker-compose
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
sudo systemctl start docker
sudo usermod -aG docker $USER
exit

# after reconnecting
git --version
docker --version
docker compose version
```

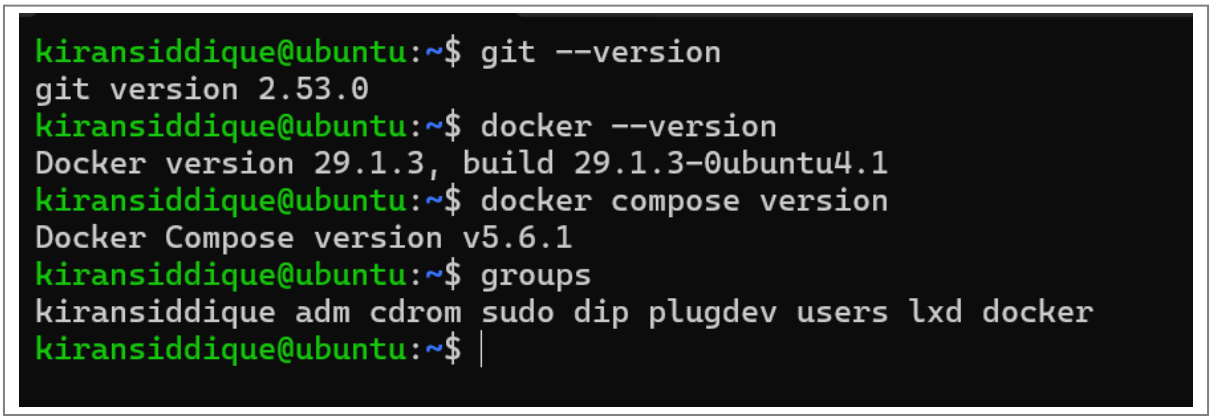


Figure 3: Git, Docker and Docker Compose versions (`docker_installation.png`*)*

Then I installed Python and prepared a virtual environment inside the Databases folder. All Python code runs directly on the Ubuntu host, and Docker runs only the databases.

```
sudo apt install -y python3 python3-venv python3-pip
cd ~/Databases
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Task 3: Start the Database Services

I started the four supplied databases (MySQL and the three MongoDB deployments for dev, test and simulated prod) with Docker Compose and checked their status. All four services show the status *healthy*.

```
docker compose up -d --wait mysql mongo-dev mongo-test mongo-prod
docker compose ps
```



Figure 4: All four database services running and healthy (`database_services.png`*)*

MongoDB 8.0 kernel issue. At first the `mongo:8.0` containers exited with an error saying that Linux kernel 6.19 and newer is not supported (SERVER-121912). The server was running kernel 7.0.0-38-generic, so MongoDB 8.0 could not start. With the instructor's permission I switched the MongoDB image to `mongo:7.0`, and the services became healthy. Because the server has limited memory, I also started the containers one at a time instead of all together.

Then I checked which host port each database uses. MySQL listens on 3307, and MongoDB dev, test and prod listen on 27017, 27018 and 27019. Python uses these host ports to reach the containers.

```
docker compose port mysql 3306
docker compose port mongo-dev 27017
docker compose port mongo-test 27017
docker compose port mongo-prod 27017
```

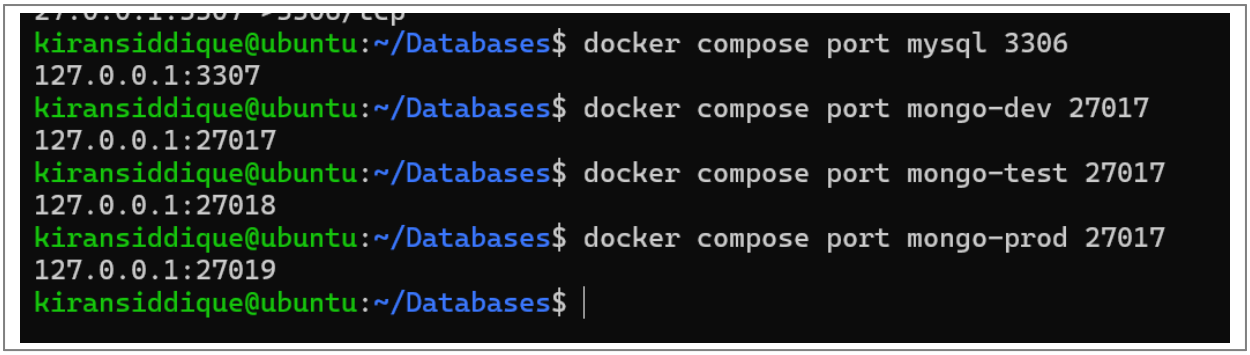


Figure 5: Host ports of the database containers (`database_ports.png`)

Task 4: MySQL with Values in the Code

The starter `db_connect.py` connects to MySQL with the username, password, host, port and database name written directly in the code. `mysql.connector.connect()` opens the connection, a small SQL query reads the stored environment name, and the cursor and connection are closed at the end. The output shows the database **school_mysql**, the server **127.0.0.1 3307** and the environment **dev**.

```
python db_connect.py
```

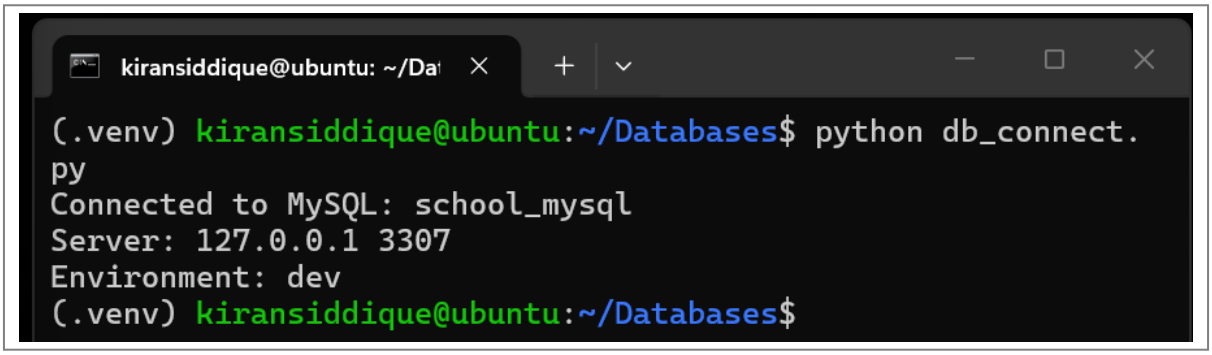


Figure 6: MySQL connection with values written in the code (`mysql_hardcoded_connection.png`)

Task 5: MongoDB with Values in the Code

I replaced the contents of the same `db_connect.py` with the MongoDB version. The username, password, endpoint and database name are stored in separate variables and joined into a MongoDB URI. `database.command("ping")` sends a real request, so the connection is actually tested. The output shows the database **school_dev**, port **27017** and environment **dev**.

```
python db_connect.py
```

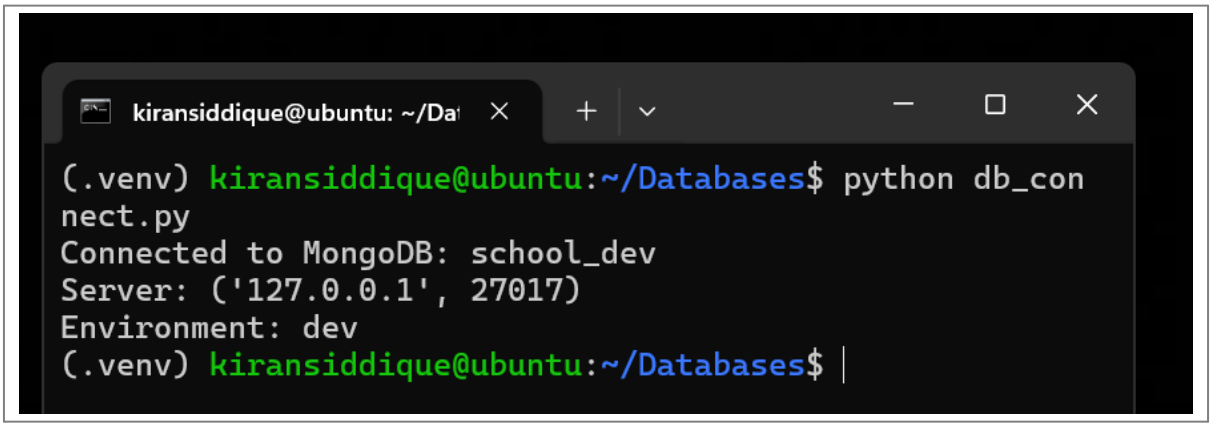


Figure 7: MongoDB connection with values written in the code (`mongo_hardcoded_connection.png`)

Task 6: MongoDB with os.getenv()

In this stage the same program reads the four settings from environment variables (`MONGO_USERNAME`, `MONGO_PASSWORD`, `MONGO_ENDPOINT` and `MONGO_DATABASE`) using `os.getenv()`. If any of them is missing, the program stops with a clear message. The connection block stayed unchanged; only the source of the settings changed. For each environment I exported the four matching values, cleared the terminal so the exports were not visible, and ran the program. The password is never printed.

```
export MONGO_USERNAME=<user>
export MONGO_PASSWORD='<dummy password>'
export MONGO_ENDPOINT=127.0.0.1:<port>
export MONGO_DATABASE=<database>
clear
python db_connect.py
```

Dev: database school_dev, port 27017, environment dev.

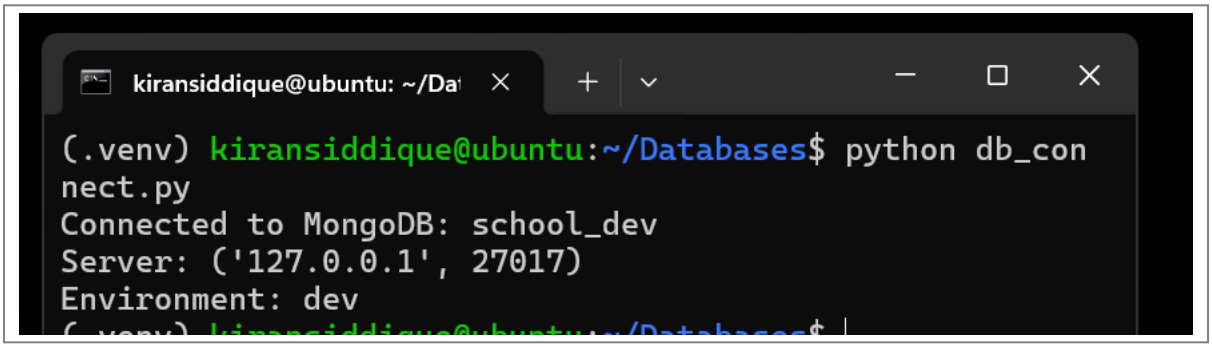


Figure 8: Environment-variable version connected to dev (`mongo_env_dev.png`)

Test: database school_test, port 27018, environment test.

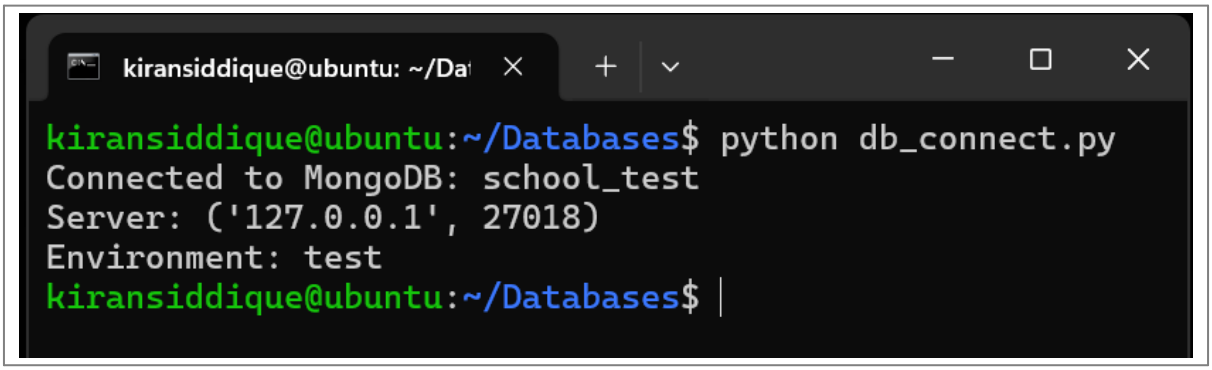


Figure 9: Environment-variable version connected to test (`mongo_env_test.png`)

Simulated prod: database school_prod, port 27019, environment prod.

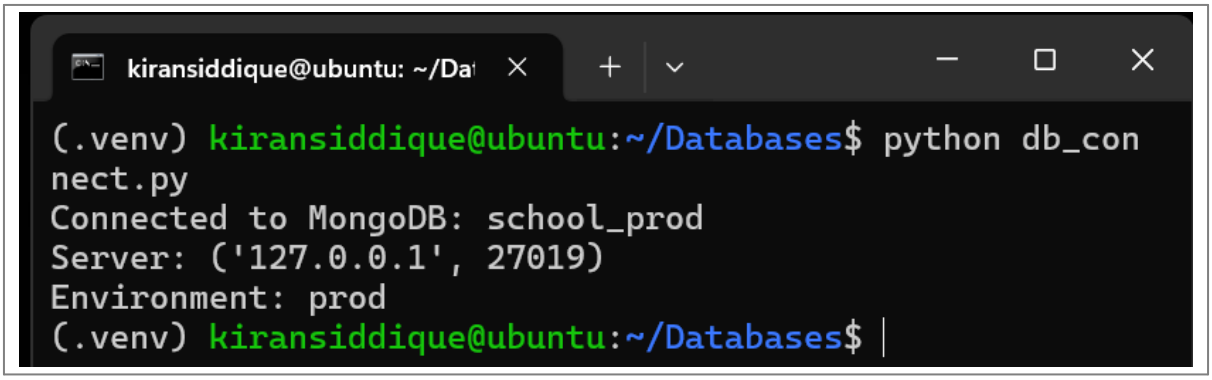


Figure 10: Environment-variable version connected to simulated prod (`mongo_env_prod.png`)

Exam Question 1 (missing environment variable). With the prod profile exported, I removed only the database name variable and ran the program again. The program stopped with the message below. The result is shown in the Exam Questions section.

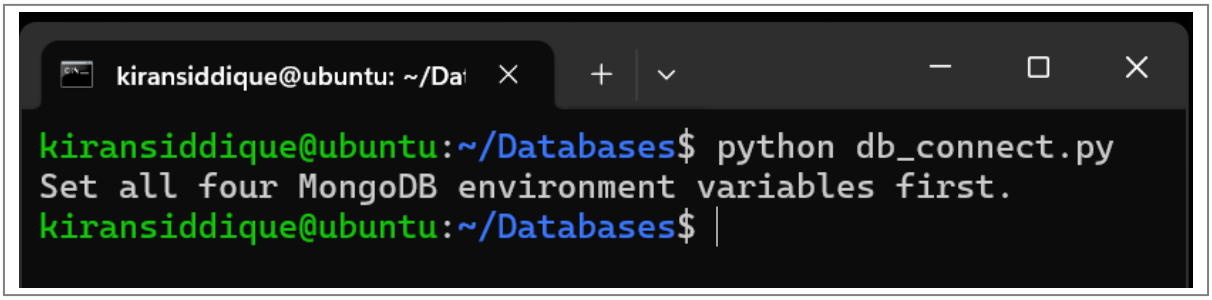


Figure 11: Q1: program stopped because MONGO_DATABASE was unset (`Q1_missing_environment.png`)

Task 7: MongoDB with a Credentials File

For the last stage I created a local `credentials.ini` from the supplied example file. It holds three sections ([dev], [test] and [prod]) with the username, password, endpoint and database name for each environment. The `cp` command printed a warning that the `-n` option is non-portable; this is harmless, and the file was created.

```
cp -n credentials.ini.example credentials.ini
ls credentials.ini
```

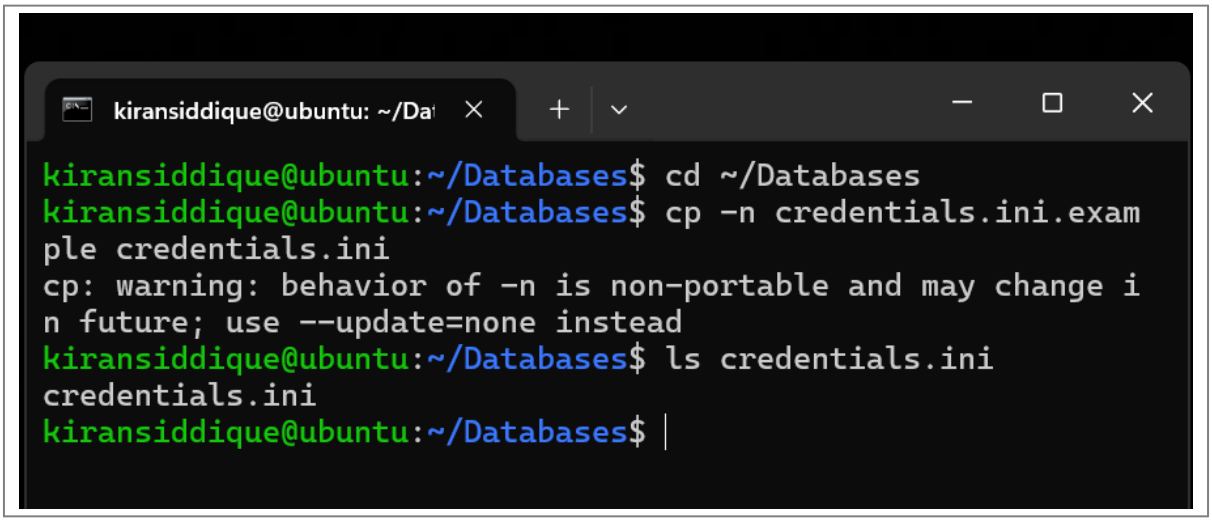


Figure 12: Credentials file created from the example (`credential_files_created.png`)

Because the file stores passwords in plain text, I restricted it to my own account with mode 600 and checked the result.

```
chmod 600 credentials.ini
stat -c '%a %n' credentials.ini
```

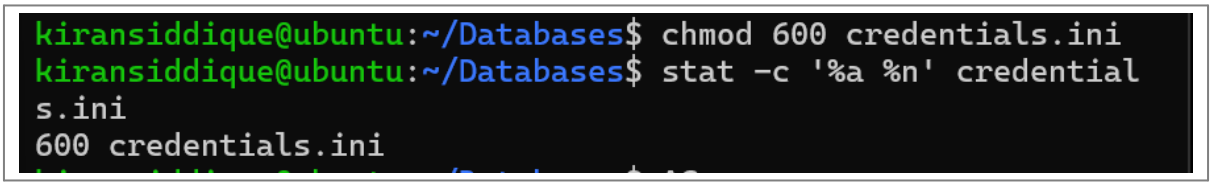


Figure 13: credentials.ini restricted to mode 600 (`credential_file_permissions.png`)

Then I replaced `db_connect.py` with the credentials-file version. It uses `configparser` to read the file and the `APP_ENV` variable to choose the section, with *dev* as the default. The MongoDB connection block is exactly the same as in the earlier stages.

```
export APP_ENV=dev     # or test, or prod
python db_connect.py
```

Dev: database school_dev, port 27017, environment dev.

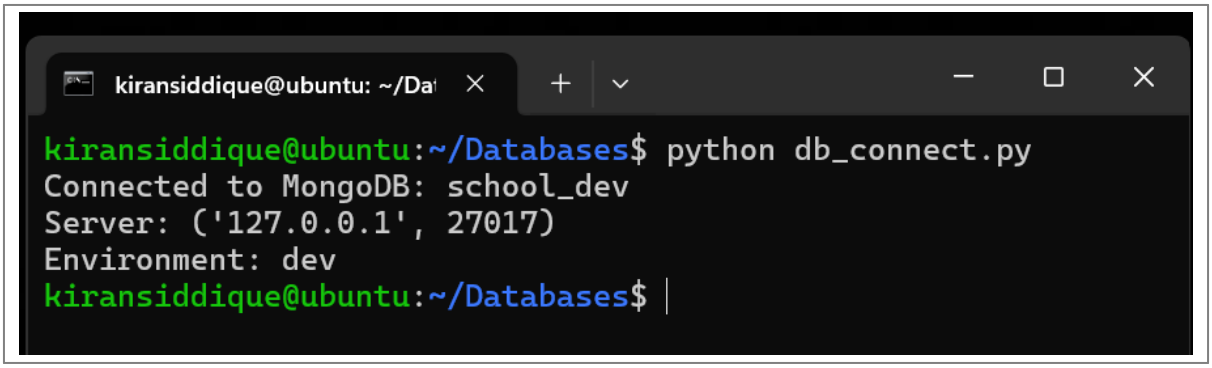


Figure 14: Credentials-file version connected to dev (`mongo_file_dev.png`)

Test: database school_test, port 27018, environment test.

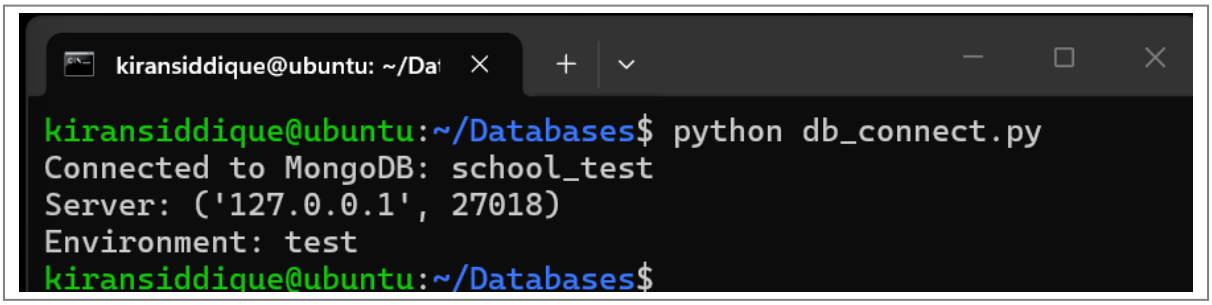


Figure 15: Credentials-file version connected to test (`mongo_file_test.png`)

Simulated prod: database school_prod, port 27019, environment prod.

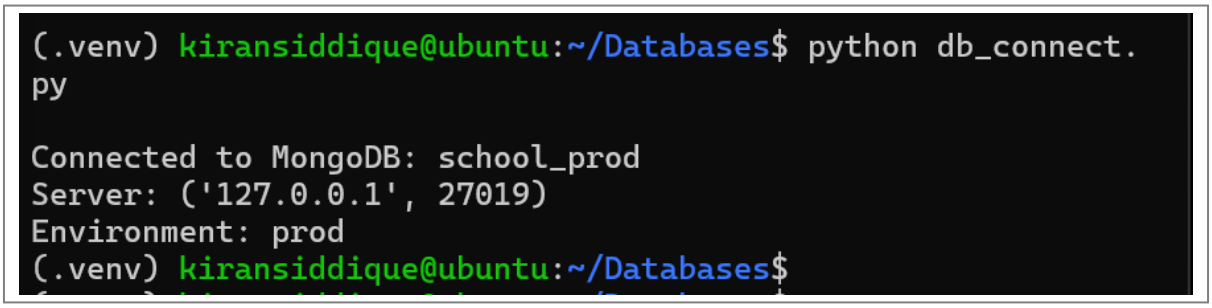


Figure 16: Credentials-file version connected to simulated prod (`mongo_file_prod.png`)

Exam Questions

Question 1: A Missing Environment Variable

With the valid prod profile exported, I ran `unset MONGO_DATABASE` and started the program (screenshot `Q1_missing_environment.png` in Task 6).

Answer: After `MONGO_DATABASE` is unset, the call `os.getenv("MONGO_DATABASE")` returns `None`. The condition `if not username or not password or not endpoint or not database_name` becomes true because `not None` is true. The program therefore raises `SystemExit` with the message "Set all four MongoDB environment variables first." and ends before it ever contacts MongoDB. After I restored `MONGO_DATABASE=school_prod`, the program connected again.

Question 2: Which Source Does the File Version Read?

I left the prod username, password, endpoint and database variables from Task 6 exported and removed only the section selector with `unset APP_ENV`. The program still connected to dev.

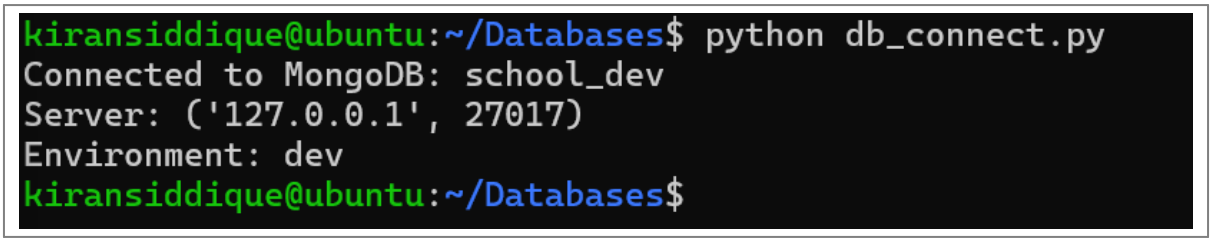


Figure 17: Q2: dev connection after unsetting APP_ENV (`Q2_file_source_selection.png`)

Answer: The file version does not read the `MONGO_*` variables at all. Once `APP_ENV` is unset, `os.getenv("APP_ENV", "dev")` returns its default value *dev*, so the [dev] section of `credentials.ini` is read. The prod settings that are still in the shell are ignored. After this I selected `APP_ENV=test` and verified that the test connection worked.

Question 3: A Wrong Password in the File

In the [test] section of my local `credentials.ini` only, I temporarily changed the password to a wrong value, kept the other settings unchanged, selected `APP_ENV=test` and ran the program. It failed with an authentication error (code 18). The file itself is not shown in the screenshot.

```
kiransiddique@ubuntu:~/Databases$ python db_connect.py 2>&1
| tail -n 4
    ^
  File "/home/kiransiddique/Databases/.venv/lib/python3.14/s
ite-packages/pymongo/helpers_shared.py", line 327, in _check
_command_response
    raise OperationFailure(errmsg, code, response, max_wire_
version)
pymongo.errors.OperationFailure: Authentication failed., ful
l error: {'ok': 0.0, 'errmsg': 'Authentication failed.', 'co
de': 18, 'codeName': 'AuthenticationFailed'}
kiransiddique@ubuntu:~/Databases$
```

Figure 18: Q3: authentication error with a wrong password (`Q3_wrong_file_password.png`)

Answer: The endpoint is only the network address of the server. It decides where the client connects, but it does not prove who the user is. MongoDB still checks the username and password after the connection is made, and a wrong password fails that check with *Authentication failed* (code 18, AuthenticationFailed), even though the endpoint and database name are correct. Afterwards I restored the supplied test password, ran the program successfully again, and set `APP_ENV=prod`. The credentials file is not committed to Git.

Summary

In this lab I edited a single Python file, `db_connect.py`, through four short connection examples. First I connected to MySQL and to MongoDB with values written in the code. Then I used `os.getenv()` to connect to the dev, test and simulated prod MongoDB deployments without changing the program, and finally I selected the same three environments from sections of a credentials file with `configparser`. For every connection I verified the target using the database name, the port and the environment value stored in the database. I also saw that a missing variable, a missing selector and a wrong password each lead to a different, predictable result. The final `db_connect.py` is the credentials-file version, and no passwords or credential-bearing URIs appear in my screenshots or answers.